Code Analysis Report: Transformer-Based Sentiment Analysis

Overview

This report provides an analysis of the sentiment analysis code project that implements a BERT-based transformer model to classify text into three sentiment categories. The code represents a complete machine learning pipeline from data preparation through model training to evaluation and inference.

Data Preparation

```
X = train.drop(columns=['sentiment'])  # Features (excluding target)
y = train['sentiment']  # Target variable

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42, stratify=y)

text = X_train["text"].tolist()
```

This section prepares the data by:

- Separating features (X) and target variable (y) from the input dataset
- Splitting the data into training and testing sets using an 80/20 split
- Stratifying the split to maintain class distribution
- Converting the text data to a list format for further processing

Model Selection and Setup

```
! pip install transformers

# from transformers import TFAutoModelForSequenceClassification
# from transformers import AutoModelForSequenceClassification
# from transformers import AutoTokenizer,AutoConfig
import numpy as np
from scipy.special import softmax

MODEL = f"cardiffnlp/twitter-roberta-base-sentiment-latest"
MODEL2 = "distilbert-base-uncased"
MODEL3 = "bert-base-uncased"

from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained(MODEL3)

def tokenize_function(text):
  return tokenizer(text, padding="max_length", truncation=True ,max_length=128
,return_tensors= "pt")

from transformers import BertForSequenceClassification
model = BertForSequenceClassification.from_pretrained(MODEL3, num_labels=3)
```

This section:

- Installs the transformers library
- Defines three potential pre-trained models for sentiment analysis
- Selects the "bert-base-uncased" model (MODEL3)

- Creates a tokenizer for the selected model
- Defines a tokenization function that pads and truncates text to 128 tokens
- Initializes the BERT model with 3 output classes for sentiment classification

Hardware Acceleration Setup

```
import torch
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model.to(device)
```

This code:

- Detects if a GPU is available for accelerated training
- Moves the model to the appropriate device (GPU or CPU)

Optimizer and Training Setup

```
import torch
from torch.optim import AdamW
optimizer = AdamW(model.parameters(), lr=5e-5)

encoded_data = tokenize_function(text)
input_ids = encoded_data['input_ids']
attention_masks = encoded_data['attention_mask']
labels = torch.tensor(y_train.values) if isinstance(y_train, pd.Series) else
torch.tensor(y_train)

from torch.utils.data import DataLoader,TensorDataset
train_dataset = TensorDataset(input_ids,attention_masks,labels)
train_dataloader = DataLoader(train_dataset, shuffle=True, batch_size=16)
```

This section:

- Initializes the AdamW optimizer with a learning rate of 5e-5
- Applies the tokenization function to convert text to model-readable format
- Extracts input IDs and attention masks from the tokenized data
- Converts labels to PyTorch tensors
- Creates a TensorDataset combining inputs, attention masks, and labels
- Sets up a DataLoader with batch size 16 and shuffling enabled

Data Verification

```
for batch in train_dataloader:
    batch_input_ids, batch_attention_masks, batch_labels = batch
    print("Input IDs shape:", batch_input_ids.shape)
    print("Attention Masks shape:", batch_attention_masks.shape)
    print("Labels shape:", batch_labels.shape)
    break
```

This code:

- Extracts and prints the shapes of the first batch from the dataloader
- Verifies that data dimensions match expectations before training

Test Data Preparation

```
test_text = X_test["text"].tolist()

encoded_test = tokenize_function(test_text)
test_input_ids = encoded_test['input_ids']
test_attention_masks = encoded_test['attention_mask']
test_labels = torch.tensor(y_test.values) if isinstance(y_test, pd.Series) else
torch.tensor(y_test)

test_dataset = TensorDataset(test_input_ids, test_attention_masks, test_labels)
test_dataloader = DataLoader(test_dataset, batch_size=16, shuffle=False)
```

This section:

- Prepares the test data using the same tokenization process as the training data
- Creates a test DataLoader without shuffling (to maintain order for evaluation)

Model Training

```
from transformers import get_linear_schedule_with_warmup

epochs = 3
total_steps = len(train_dataloader) * epochs
scheduler = get_linear_schedule_with_warmup(optimizer, num_warmup_steps=0,
num_training_steps=total_steps)

for epoch in range(epochs):
  model.train()
  for batch in train_dataloader:
    batch_input_ids, batch_attention_masks, batch_labels = [b.to(device) for b
in batch]
    outputs = model(batch_input_ids, attention_mask=batch_attention_masks,
labels=batch_labels)
    loss = outputs.loss
    loss.backward()
    optimizer.step()
    scheduler.step()
    optimizer.zero_grad()
  print(f"Epoch {epoch + 1}/{epochs}, Loss: {loss.item()}")
```

This code:

- Sets up a learning rate scheduler with linear warmup
- Trains the model for 3 epochs
- For each batch:
    - Moves data to the appropriate device
    - Computes forward pass and loss
    - Performs backpropagation
    - Updates model weights with optimizer
    - Updates learning rate with scheduler
    - Resets gradients
- Prints the loss value after each epoch

Model Evaluation (Simple)

```
model.eval()
predictions, true_labels = [], []
model.eval()
with torch.no_grad():
    for batch in test_dataloader:
        input_ids, attention_mask, labels = [b.to(device) for b in batch]
        outputs = model(input_ids, attention_mask=attention_mask)
        logits = outputs.logits
        predictions.extend(torch.argmax(logits, dim=-1).cpu().numpy())
        true_labels.extend(labels.cpu().numpy())

from sklearn.metrics import accuracy_score, f1_score
print("Accuracy:", accuracy_score(true_labels, predictions))
print("F1 Score:", f1_score(true_labels, predictions, average='weighted'))
```

This section:

- Sets the model to evaluation mode
- Disables gradient calculation for efficiency
- Processes each test batch and collects predictions
- Uses scikit-learn to calculate accuracy and F1 score

Comprehensive Evaluation

```
import torch
from sklearn.metrics import accuracy_score, f1_score
from tqdm import tqdm

def evaluate(model, dataloader, device):
    model.eval()
    predictions, true_labels = [], []
    total_loss = 0.0
    with torch.no_grad():
        for batch in tqdm(dataloader, desc="Evaluating"):
            input_ids, attention_mask, labels = [b.to(device) for b in batch]
            outputs = model(input_ids, attention_mask=attention_mask,
labels=labels)
            logits = outputs.logits
            loss = outputs.loss
            total_loss += loss.item()
            predictions.extend(torch.argmax(logits, dim=1).cpu().numpy())
            true_labels.extend(labels.cpu().numpy())

    avg_loss = total_loss / len(dataloader)
    accuracy = accuracy_score(true_labels, predictions)
    f1 = f1_score(true_labels, predictions, average='weighted')
    return avg_loss, accuracy, f1, predictions, true_labels

test_loss, test_accuracy, test_f1, test_predictions, test_true_labels =
evaluate(model, test_dataloader, device)
print(f"Test Loss: {test_loss:.4f}, Accuracy: {test_accuracy:.4f}, F1 Score:
{test_f1:.4f}")
```

This code:

- Implements a more comprehensive evaluation function
- Adds progress bar visualization with tqdm
- Calculates and returns average loss, accuracy, F1 score, and prediction arrays
- Prints final metrics in a formatted string

Detailed Classification Analysis

```
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(test_true_labels, test_predictions)
print("Confusion Matrix:")
print(cm)

plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Negative',
'Positive','Neutral'], yticklabels=['Negative', 'Positive','Neutral'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix')
plt.show()

print("\nClassification Report:")
print(classification_report(test_true_labels, test_predictions,
target_names=['Negative', 'Positive','Neutral']))
```

This section:

- Generates a confusion matrix to visualize classification performance
- Creates a heatmap visualization of the confusion matrix
- Produces a detailed classification report with precision, recall, and F1 for each class

Inference Function

```
def predict_sentiment(text, model, tokenizer, device, label_mapping=None):
    model.eval()
    encoded = tokenizer(
        text,
        padding='max_length',
        max_length=128,
        truncation=True,
        return_tensors='pt'
    )
    input_ids = encoded['input_ids'].to(device)
    attention_mask = encoded['attention_mask'].to(device)

    with torch.no_grad():
        outputs = model(input_ids, attention_mask=attention_mask)
        logits = outputs.logits
        prediction = torch.argmax(logits, dim=1).cpu().numpy()[0]

    if label_mapping:
        reverse_mapping = {v: k for k, v in label_mapping.items()}
        return reverse_mapping.get(prediction, prediction)
    return prediction
```

This function:

- Takes a text input and returns its sentiment prediction
- Handles tokenization and model inference
- Supports label mapping to convert numeric predictions to text labels
- Performs inference efficiently with gradient calculation disabled

Example Usage

```
label_mapping = {'negative': 0, 'neutral': 1, 'positive': 2}
test_examples = [
    "I absolutely love this product, it's amazing!",
    "This is the worst experience I've ever had.",
    "The movie was okay, nothing special.",
    "I'm so happy with my purchase!",
    "I hate how slow this service is."
]

for text in test_examples:
    predicted_label = predict_sentiment(text, model, tokenizer, device,
label_mapping)
    print(f"Text: {text}")
    print(f"Predicted Sentiment: {predicted_label}\n")
```

This code:

- Defines a mapping from numeric labels to sentiment categories
- Creates test examples with different sentiment expressions
- Runs each example through the sentiment prediction function
- Prints both the input text and predicted sentiment

Model Persistence

```
model.save_pretrained("sentiment_model")
tokenizer.save_pretrained("sentiment_model")

def predict_sentiment(text):
    inputs = tokenizer(text, return_tensors="pt", truncation=True, padding=True,
max_length=128)
    inputs = {key: val.to(device) for key, val in inputs.items()}
    with torch.no_grad():
        outputs = model(**inputs)
    return torch.argmax(outputs.logits, dim=1).item()
```

This section:

- Saves the trained model and tokenizer for future use
- Includes a simplified prediction function for inference after loading

Summary

The code implements a complete transformer-based sentiment analysis system using BERT. It covers the entire machine learning workflow:

1. Data preparation and splitting
2. Model selection and initialization
3. Training with optimization techniques
4. Comprehensive evaluation using multiple metrics
5. Visualization of results
6. Inference functionality for practical use
7. Model persistence for future applications

The implementation leverages the powerful Hugging Face Transformers library along with PyTorch to create a system capable of distinguishing between negative, neutral, and positive sentiment in text.